Smart Manufacturing IoT-Based Predictive Maintenance and Anomaly Detection

Group 22
Aomsin Nimsombun, Colin Xiong, Kaoklong Panitpongpun, Daniel Benjamin
MANU 465 – University of British Columbia

1 Introduction

1.1 Problem Statement and Motivation

Modern smart manufacturing systems rely on IoT sensors to continuously monitor equipment conditions. Despite rich sensor data—including temperature, vibration, pressure, and humidity—many facilities depend on reactive or scheduled maintenance strategies that result in unexpected breakdowns, unnecessary servicing, and significant operational costs. Studies estimate that unplanned downtime costs industrial manufacturers hundreds of thousands of dollars per hour, making even marginal improvements in early failure detection highly valuable [1].

This project develops and evaluates machine learning models for predicting whether maintenance is required based on IoT sensor data. Additionally, unsupervised anomaly detection is explored to identify abnormal machine behavior prior to failure events, addressing scenarios where labeled failure data may be unavailable. The goal is to demonstrate how data-driven models can support the transition from reactive to proactive maintenance scheduling in smart manufacturing.

1.2 Project Scope

This project applies five ML techniques of increasing complexity: (1) **Logistic Regression** as an interpretable baseline; (2) **Random Forest** for non-linear feature interactions; (3) **ANN** with TensorFlow/Keras for complex pattern recognition; (4) **PCA** for dimensionality reduction; and (5) **Isolation Forest** for unsupervised anomaly detection. All models are evaluated with emphasis on precision-recall trade-offs relevant to maintenance, where missed failures carry higher cost than false alarms.

2 Dataset Description

This project utilizes the *Smart Manufacturing IoT-Cloud Monitoring Dataset* from Kaggle, simulating real-time sensor measurements from 50 industrial machines.

Table 1: Dataset overview.

Property	**Description**
Number of records	100,000
Number of machines	50 unique machines
Time resolution	1-minute intervals
Time range	69 days (Jan–Mar 2025)
Primary sensors	Temperature, Vibration, Pressure, Humidity
Additional features	Energy Consumption, Downtime Risk Score
Operational states	Idle (0), Running (1), Failure (2)
Target variable	`maintenance_required` (binary: 0/1)
Missing values	None

The dataset contains 13 columns in total. Non-predictive identifiers (`timestamp`, `machine_id`) were removed during preprocessing, and `failure_type` was label-encoded.

2.1 Exploratory Data Analysis

The class distribution is imbalanced: 80.3% no-maintenance vs. 19.7% maintenance-required. As shown in Figure 1, `temperature` has the strongest individual correlation with the target ($r = 0.283$), followed by `vibration` ($r = 0.113$). Weak individual correlations for other features suggest non-linear combinations are important—confirmed by the superior performance of non-linear models. No missing values were found.

Figure 1: Correlation matrix of sensor features and maintenance target.

3 Methodology

3.1 Data Preprocessing

Preprocessing involved five steps: (1) dropping non-predictive identifiers (`timestamp`, `machine_id`); (2) label-encoding `failure_type`; (3) retaining 10 features—five sensor measurements, two operational indicators (`machine_status`, `anomaly_flag`), and three derived features (`predicted_remaining_life`, `downtime_risk`, `failure_type_encoded`); (4) stratified 80/20 train-test split (80,000/20,000 samples) preserving class proportions; and (5) standardizing features ($z = \frac{x-\mu}{\sigma}$) using `StandardScaler` fitted *only* on training data to prevent information leakage. Class imbalance (80.3% vs. 19.7%) was addressed via `class_weight='balanced'`, which inversely weights classes by frequency so the minority class receives higher importance during training.

3.2 Machine Learning Models

3.2.1 Model 1: Logistic Regression (Baseline)

Logistic Regression was chosen as the interpretable baseline because of its well-understood linear decision boundary and probabilistic output. It models the probability of maintenance as a sigmoid function of weighted input features: $P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b)$. The L-BFGS solver was used with balanced class weights and max 1000 iterations. Five-fold cross-validation with F1-score (rather than accuracy, which is misleading for imbalanced data) assessed generalization stability.

3.2.2 Model 2: Random Forest with Hyperparameter Tuning

Random Forest was selected to capture non-linear feature interactions that a linear model cannot represent. It aggregates predictions from many decision trees, each trained on a bootstrap sample with random feature subsets, reducing overfitting through bagging [5]. Hyperparameter tuning via `GridSearchCV` (3-fold CV) searched over `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{10, 20, \text{None}\}$, and `min_samples_split` $\in \{2, 5\}$, selecting the best configuration by F1-score. Feature importance was extracted via Gini impurity to provide actionable insights into which sensors most strongly predict maintenance needs.

3.2.3 Model 3: Artificial Neural Network (TensorFlow/Keras)

A feed-forward ANN was built using TensorFlow/Keras [3] to test whether deeper non-linear modeling improves on the ensemble. The funnel architecture compresses representations: input (10) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Dense(16, ReLU) $\rightarrow$ Dense(1, sigmoid), totalling 3,329 parameters. Training used Adam (lr=0.001), binary crossentropy, batch size 256, 20% validation split, balanced class weights, and **Early Stopping** (patience=10) monitoring validation loss with best-weight restoration—preventing overfitting by halting training when validation performance stops improving.

3.2.4 Dimensionality Reduction: PCA

PCA was applied to evaluate the intrinsic dimensionality of the feature space by computing eigenvectors of the covariance matrix and projecting data onto directions of maximum variance. A scree plot and cumulative variance plot determined the components needed for 95% variance retention. To quantify the practical impact of dimensionality reduction, a Logistic Regression model trained on PCA-reduced features was compared to the full-feature baseline.

3.2.5 Anomaly Detection: Isolation Forest

Isolation Forest [6], an unsupervised algorithm, isolates anomalies by random partitioning—points requiring fewer partitions are anomalies. The contamination parameter was set to 0.2 (≈observed maintenance rate). Performance was evaluated against true labels for comparison with supervised methods.

3.3 Evaluation Metrics

All models were evaluated on the held-out test set using accuracy, precision, recall, F1-score ($F_1 = 2 \cdot \frac{P \times R}{P+R}$), ROC-AUC, and confusion matrices. In predictive maintenance, recall is particularly important because missed failures can lead to unplanned downtime with higher cost than false alarms.

4 Results

4.1 Model Performance Summary

Table 2 presents the test set performance of all models. Exact metric values are reproduced from the accompanying Jupyter notebook (`code.ipynb`).

Table 2: Model performance comparison on the test set.

Model	**Accuracy**	**Precision**	**Recall**	**F1**	**ROC-AUC**
Naive Baseline	∼0.80	—	—	—	—
Logistic Regression	0.9840	1.0000	0.9190	0.9578	0.9903
Random Forest (tuned)	1.0000	1.0000	1.0000	1.0000	1.0000
Neural Network (ANN)	0.9979	0.9914	0.9980	0.9947	1.0000
LR + PCA (8 PCs)	0.9775	1.0000	0.8855	0.9393	0.9826
Isolation Forest	0.9381	0.8335	0.8566	0.8449	0.9604

4.2 Logistic Regression

Logistic Regression achieved accuracy of 0.9840, perfect precision (1.0000), recall of 0.9190, F1 of 0.9578, and ROC-AUC of 0.9903. It never issued a false alarm but missed ∼8% of true maintenance cases (319 false negatives out of 3,939 positive samples). Cross-validation F1 scores (0.9578±0.0051)

showed very low variance, indicating stable generalization across folds. The perfect precision suggests the linear decision boundary is conservative—it only flags maintenance when highly confident, at the expense of missing some true cases.

4.3 Random Forest

After GridSearchCV (best: `n_estimators`=100, `max_depth`=10, `min_samples_split`=2), Random Forest achieved perfect scores across all metrics (1.0000). The jump from LR (F1 = 0.9578) to RF (F1 = 1.0000) quantifies how much predictive information resides in non-linear feature interactions. However, perfect classification warrants scrutiny: it likely reflects the presence of derived features (`anomaly_flag`, `downtime_risk`) that may encode the target variable, rather than indicating the model would generalize perfectly to raw sensor data. Feature importance (Figure 2) identified `machine_status` (0.362), `predicted_remaining_life` (0.248), `failure_type_encoded` (0.138), and `anomaly_flag` (0.130) as the top four predictors, accounting for over 85% of total importance. Notably, the top four features are all derived/operational features rather than raw sensor readings, reinforcing the leakage concern.

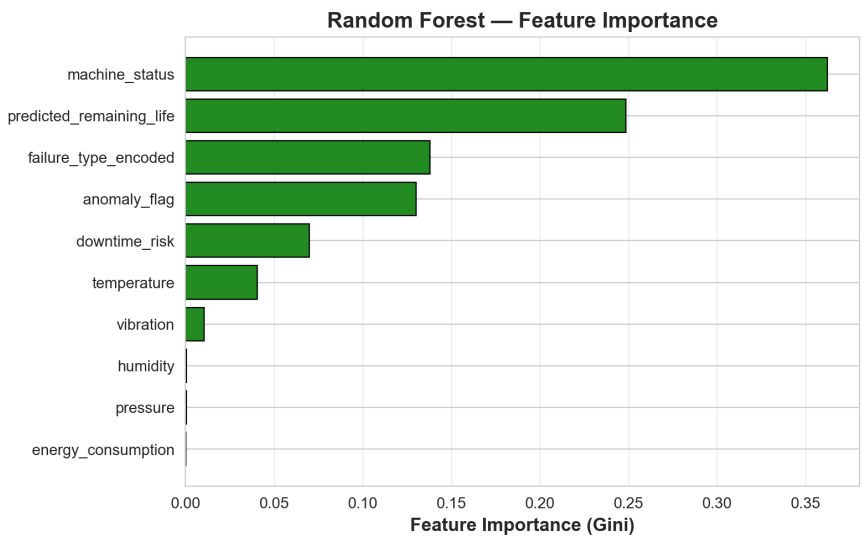

Figure 2: Random Forest feature importance ranking by Gini impurity.

4.4 Neural Network (ANN)

The ANN achieved accuracy of 0.9979, precision of 0.9914, recall of 0.9980, F1 of 0.9947, and ROC-AUC of 1.0000. Compared to LR, it improved recall from 0.9190 to 0.9980 (reducing missed maintenance cases from 319 to only 8), at the cost of a slight precision decrease to 0.9914 (34 false alarms). This precision-recall trade-off is favorable for maintenance: missing 8 failures is far less costly than 34 unnecessary inspections. Figure 3 shows smooth convergence with final validation accuracy of 0.9972 and validation loss of 0.0067. Early Stopping halted training after ∼55 epochs, well before the 100-epoch maximum, confirming effective regularization.

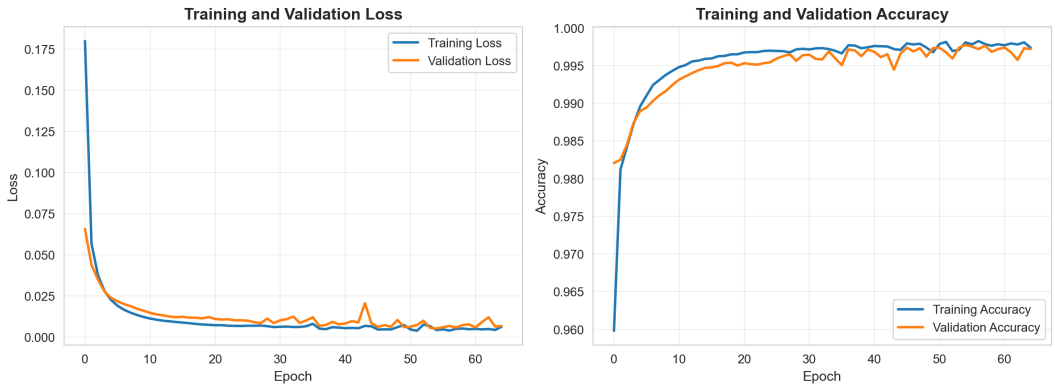

Figure 3: ANN training and validation curves showing convergence behavior.

4.5 PCA Analysis

PCA revealed that 8 of 10 components are needed to retain 95% of total variance, indicating only moderate redundancy. PC1 explains 26.0% and PC2 explains 15.3% individually. LR trained on the 8 PCA components achieved F1 of 0.9393 and ROC-AUC of 0.9826—a modest decrease from full-feature LR (F1 = 0.9578, ΔF1 = -0.0185). This confirms that some discriminative information resides in the two discarded components, and that dimensionality reduction from 10 to 8 features provides minimal compression benefit for this dataset.

4.6 Isolation Forest

The unsupervised Isolation Forest achieved accuracy of 0.9381, precision of 0.8335, recall of 0.8566, F1 of 0.8449, and ROC-AUC of 0.9604. Despite 674 false positives and 565 false negatives, the respectable ROC-AUC demonstrates viability when labeled data is scarce.

4.7 ROC Curve Comparison

Figure 4 compares ROC curves for all models. All supervised models perform well above the random baseline. The threshold-independent ROC comparison is valuable in practice, where the decision threshold can be tuned based on the cost ratio of missed failures to false alarms.

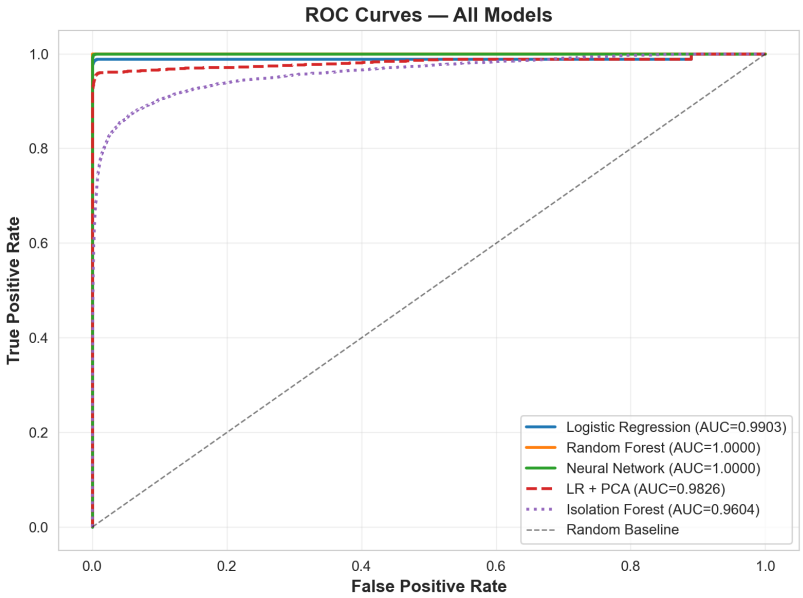

Figure 4: ROC curves for all models compared on the test set.

5 Discussion

5.1 Strengths

- **Comprehensive methodology:** Five ML techniques spanning linear models, ensembles, deep learning, dimensionality reduction, and unsupervised anomaly detection, providing a thorough comparison across model families.
- **Rigorous evaluation:** Stratified splitting, cross-validation, multiple metrics (accuracy, precision, recall, F1, ROC-AUC), and a naive baseline ensure models learn beyond trivial predictions.
- **Data leakage prevention:** `StandardScaler` fitted only on training data; test set not seen during training or tuning.
- **Systematic tuning:** GridSearchCV for Random Forest and Early Stopping for the ANN both prevent overfitting through principled mechanisms.
- **Actionable insights:** Feature importance analysis identified the most predictive sensors, providing direct value for maintenance engineers.

5.2 Limitations

- **Simulated data:** The dataset is synthetically generated and may not capture real-world noise, sensor drift, missing readings, and operational complexity.
- **Potential data leakage:** Features such as `downtime_risk`, `anomaly_flag`, and `predicted_remaining_life` may encode the target variable, inflating performance—particularly Random Forest's perfect scores. These features may not be available at prediction time in a real deployment.

- **No temporal modeling:** All models treat each record independently, ignoring sequential degradation patterns and time-series structure.
- **Single dataset:** Generalizability to different manufacturing environments and sensor configurations remains unvalidated.

5.3 Potential Improvements

- **Temporal modeling:** LSTM or GRU networks could exploit sequential sensor degradation trends, potentially improving predictions on raw sensor data.
- **Feature engineering:** Rolling window statistics (mean, standard deviation, rate of change) and per-machine baselines could enrich the feature space.
- **Leakage investigation:** Retraining without derived features would clarify how much performance comes from raw sensors alone.
- **Explainability:** SHAP values could provide instance-level feature attribution, helping engineers understand *why* specific predictions were made.

6 Conclusion

This project applied five ML techniques to a 100,000-record IoT dataset: Logistic Regression (F1 = 0.9578), Random Forest (F1 = 1.0000), TensorFlow/Keras ANN (F1 = 0.9947), PCA, and Isolation Forest (F1 = 0.8449). The ANN achieved the best precision-recall balance, missing only 8 maintenance events. PCA confirmed moderate redundancy (8 of 10 components for 95% variance). The primary limitations are the synthetic dataset, potential feature leakage, and absence of temporal modeling. Future work should explore recurrent architectures, feature auditing, and real-world validation.

References

[1] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Butterworth-Heinemann, 2002.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, 2016, pp. 265–283.

[4] "Smart Manufacturing IoT-Cloud Monitoring Dataset," Kaggle. [Online]. Available: `https://www.kaggle.com/datasets`

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.